WEEK 5

PAYMENT TRANSACTION MANAGEMENT

Introduction

The Payment table is used to store and manage the payment transaction details of customers. It maintains information such as payment ID, order ID, payment date, payment mode, payment status, and transaction amount.

The Payment table helps the system to track successful, failed, and pending transactions related to customer orders.

Create Payment Table

Payment Entity

The **Payment** entity stores the payment transaction details associated with customer orders.

Attributes of Payment Table

Attribute	Description
Payment_ID	Unique payment identifier
Order_ID	Reference of the order
Payment_Date	Date of payment transaction
Payment_Mode	Mode of payment such as UPI, Card, Net Banking or Cash
Payment_Status	Status of payment such as Success, Failed or Pending
Transaction_Amount	Amount paid by the customer

```
SQL> CREATE TABLE Payment(Payment_ID VARCHAR(10),Order_ID VARCHAR(20),Payment_Date DATE,Payment_Mode VARCHAR(20),Payment_Status VARCHAR(10),Transact
ion_Amount NUMBER(10,2));

Table created.
```

Apply Constraints

Constraints are used to maintain the accuracy and integrity of data in the Payment table.

Primary Key

`Payment_ID` is used as the primary key because each payment should have a unique identification number.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT PK_Payment PRIMARY KEY (Payment_ID);

Table altered.
```

Foreign Key

`Order_ID` is used as a foreign key to establish a relationship between the Payment table and Orders table.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT FK_Payment_Order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);

Table altered.
```

NOT NULL Constraint

Payment mode and status should not be empty.

```
SQL> ALTER TABLE Payment MODIFY (Payment_Mode CONSTRAINT NN_Payment_Mode NOT NULL);

Table altered.

SQL> ALTER TABLE Payment MODIFY (Payment_Status CONSTRAINT NN_Payment_Status NOT NULL);

Table altered.
```

CHECK Constraint

The transaction amount should always be greater than zero.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_Transaction_Amount CHECK (Transaction_Amount > 0);

Table altered.
```

Establish Relationship with Orders

The Payment table is related to the Orders table using the `Order_ID` attribute.

Order and Payment Relationship

- Each order can have one payment transaction.
- Each payment belongs to one order.

Relationship

Order (1) -------- (1) Payment

The `Order_ID` in the Payment table acts as a **Foreign Key** and refers to the `Order_ID` in the Orders table.

This relationship helps to identify which payment belongs to which customer order.

Insert Payment Values

The following payment records are inserted into the Payment table.

```
SQL> INSERT INTO Payment VALUES ('P101', 111, TO_DATE('11-07-2026','DD-MM-YYYY'), 'UPI', 'Success', 2500.00);

1 row created.

SQL> INSERT INTO Payment VALUES ('P102', 112, TO_DATE('12-07-2026','DD-MM-YYYY'), 'Card', 'Success', 1800.00);

1 row created.

SQL> INSERT INTO Payment VALUES ('P103', 113, TO_DATE('13-07-2026','DD-MM-YYYY'), 'Net Banking', 'Pending', 3200.00);

1 row created.

SQL> INSERT INTO Payment VALUES ('P104', 114, TO_DATE('14-07-2026','DD-MM-YYYY'), 'Cash', 'Success', 1500.00);

1 row created.

SQL> INSERT INTO Payment VALUES ('P105', 115, TO_DATE('15-07-2026','DD-MM-YYYY'), 'UPI', 'Failed', 2200.00);

1 row created.
```

Display Payment Table

```
SQL> SELECT PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT FROM Payment;

PAYMENT_ID ORDER_ID            PAYMENT_DATE PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT
---------- ------------------- ------------ --------------- --------------- ------------------
P101       111                 11-JUL-26    UPI             Success   2500.00
P102       112                 12-JUL-26    Card            Success   1800.00
P103       113                 13-JUL-26    Net Banking     Pending   3200.00
P104       114                 14-JUL-26    Cash            Success   1500.00
P105       115                 15-JUL-26    UPI             Failed    2200.00
```

SQL Queries for Payment Management

Query 1: Display All Successful Payments

This query is used to display all the payment transactions whose status is **Success**.

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';

PAYMENT_ID ORDER_ID            PAYMENT_DATE PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT
---------- ------------------- ------------ --------------- --------------- ------------------
P101       111                 11-JUL-26    UPI             Success   2500.00
P102       112                 12-JUL-26    Card            Success   1800.00
P104       114                 14-JUL-26    Cash            Success   1500.00
```

Query 2: Find Failed Transactions

This query identifies all the payment transactions that have failed.

```
SQL> SELECT * FROM PAYMENT WHERE PAYMENT_STATUS = 'Failed';

PAYMENT_ID ORDER_ID            PAYMENT_DATE PAYMENT_MODE    PAYMENT_STATUS  TRANSACTION_AMOUNT
---------- ------------------- ------------ --------------- --------------- ------------------
P105       115                 15-JUL-26    UPI             Failed    2200.00
```

Query 3: Count Total Successful and Failed Payments

This query is used to count the number of successful and failed payment transactions.

```
SQL> SELECT PAYMENT_STATUS, COUNT(*) AS TOTAL_PAYMENTS FROM PAYMENT WHERE PAYMENT_STATUS IN ('Success','Failed') GROUP BY PAYMENT_STATUS;

PAYMENT_STATUS  TOTAL_PAYMENTS
--------------- --------------
Failed                       1
Success                      3
```

Analyze Payment Methods Used by customers

Report 1: Payment mode Analysis

Find

- Number of transactions Using UPI
- Number of card payment
- Most preferred payment method

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE PAYMENT_MODE = 'UPI';

UPI_TRANSACTIONS
----------------
               2

SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM PAYMENT WHERE PAYMENT_MODE = 'Card';

CARD_PAYMENTS
-------------
            1

SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;

PAYMENT_MODE         TRANSACTION_COUNT
-------------------- -----------------
UPI                                  2
Cash                                 1
Net Banking                          1
Card                                 1

SQL> SELECT PAYMENT_MODE, TRANSACTION_COUNT FROM (SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRA
NSACTION_COUNT DESC) WHERE ROWNUM = 1;

PAYMENT_MODE         TRANSACTION_COUNT
-------------------- -----------------
UPI                                  2
```

Report 2: Revenue Analysis

Find:

- Total revenue generated.
- Revenue by payment method.
- Average transaction amount

```
SQL> SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';

TOTAL_REVENUE
-------------
         8000

SQL> SELECT PAYMENT_MODE, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'Success' GROUP BY PAYMENT_MODE ORDER BY TOTA
L_REVENUE DESC;

PAYMENT_MODE         TOTAL_REVENUE
-------------------- -------------
UPI                           4700
Card                          1800
Cash                          1500

SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVERAGE_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                      2000
```

Report 3: Customer Payment history

Display:

- Customer Name
- Order ID
- Payment mode
- Amount
- Payment status

```
SQL> SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, P.PAYMENT_DATE, P.PAYMENT_MODE, P.TRANSACTION_AMOUNT AS AMOUNT, P.PAYMENT_STATUS FROM CUSTOME
R C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID ORDER BY P.PAYMENT_DATE;

CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- --------------------
ORDER_DAT PAYMENT_D PAYMENT_MODE             AMOUNT PAYMENT_ST
--------- --------- -------------------- ---------- ----------
Arun Kumar Raj                                     111
10-JAN-25 11-JUL-26 UPI                        2500 Success

Priya Sharma                                       112
15-FEB-25 12-JUL-26 Card                       1800 Success

Rahul Raj                                          113
20-MAR-25 13-JUL-26 Net Banking                3200 Pending


CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- --------------------
ORDER_DAT PAYMENT_D PAYMENT_MODE             AMOUNT PAYMENT_ST
--------- --------- -------------------- ---------- ----------
Divya Devi                                         114
05-APR-25 14-JUL-26 Cash                       1500 Success
```

Payment Analysis:

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE, ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACT
ION FROM PAYMENT WHERE PAYMENT_STATUS = 'Success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;

PAYMENT_MODE    TRANSACTION_COUNT TOTAL_REVENUE AVG_TRANSACTION
--------------- ----------------- ------------- ---------------
UPI                             2          4700            2350
Card                            1          1800            1800
Cash                            1          1500            1500
```